

LC 407: Trapping Rain Water - II

Comprehensive Study Notes & Implementation

Goldman Sachs

Qualcomm

Problem Visualization

This problem uses a **2D graph** where each row represents elevation in a 2D map (see LeetCode official for visualization).

Example 1: Basic Water Trapping

```
5 5 5 5 → boundaries
5 1 2 5
5 2 1 5
5 5 5 5
```

Inside heights: 1, 2, 2, 1

Water trapped: $(5-1) + (5-2) + (5-2) + (5-1) = 14$

Example 2: Minimum Surrounding Wall

```
10 10 10
10  1 10
10 (2) 10 ← water will leak from here
```

Water escapes through the lowest boundary first, exactly like min distance first.

The amount of water depends on the minimum surrounding wall.

Core Algorithm Logic

- **Step 1:** Push every boundary cell into a min-heap. Boundaries can never trap water. Mark boundaries as visited.
- **Step 2:** Always remove the minimum height = currheight.
- **Step 3:** Visit neighbours.

Case 1: Higher Neighbour

Let curr = 5, next = 8
→ New boundary = 8
→ **No water trapped**
→ currheight = newheight

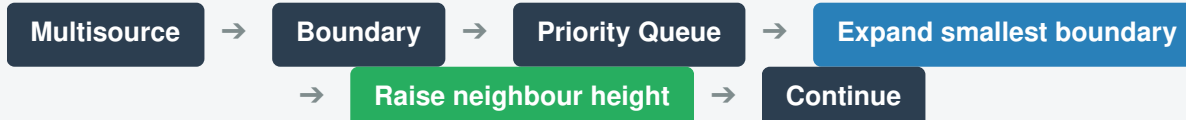
Case 2: Lower Neighbour

Let curr = 5, next = 2
→ Water filled = $5 - 2 = 3$
→ New effective height = 5 (because water filled it, push 5 in heap)

Heap stores: {height, {row, col}}

Time Complexity (TC): Each cell enters once on heap so $O(NM \log(NM))$

Execution Flow & Implementation Steps



S1: Define directions (up, down, left, right)

S2: Initialize Priority Queue (Min-Heap)

S3: Initialize seen (visited array)

S4 & S5: Push first & last columns, and first & last rows

S6: Pop min elements iteratively

S7: Make cases for neighbours

C++ Source Code

```
class Solution {
public:
    int trapRainWater(vector<vector<int>>& heightMap) {
        int n = heightMap.size();
        int m = heightMap[0].size();

        // pq will store {heights, {row, col}}
        priority_queue<pair<int, pair<int, int>>, vector<pair<int, pair<int, int>>>, greater<pair<int, pair<int, int>>>>
        vector<pair<int, int>> dir = {{-1, 0}, {0, -1}, {0, 1}, {1, 0}};

        vector<vector<int>> seen(n, vector<int>(m, 0));

        // push boundary elements to heap
        for(int j = 0; j < m; j++){
            pq.push({heightMap[0][j], {0, j}});
            seen[0][j] = 1;
        }
    }
};
```

```

        pq.push({heightMap[n-1][j], {n-1, j}});
        seen[n-1][j] = 1;
    }

    for(int i = 1; i < n-1; i++){ // corners already kr diye
        pq.push({heightMap[i][0], {i, 0}});
        seen[i][0] = 1;

        pq.push({heightMap[i][m-1], {i, m-1}});
        seen[i][m-1] = 1;
    }

    int ans = 0;
    while(!pq.empty()){
        int currh = pq.top().first;
        int r = pq.top().second.first;
        int c = pq.top().second.second;
        pq.pop();

        for(auto &it : dir){
            int nr = r + it.first;
            int nc = c + it.second;

            if(nr >= 0 && nc >= 0 && nr < n && nc < m && !seen[nr][nc]){
                int nh = heightMap[nr][nc];
                if(currh > nh){
                    ans += currh - nh;
                    pq.push({currh, {nr, nc}}); // currh hi push krenges as paani se bhar
                    seen[nr][nc] = 1;
                } else {
                    pq.push({nh, {nr, nc}});
                    seen[nr][nc] = 1;
                }
            }
        }
    }

    return ans;
}
};

```